

Lecture Scribe CSE400



L10_S1.pdf

PDF

Create a detailed lecture scribe from the attached context to be used as exam reference material. For every definition, theorem, proof, and worked example, explicitly write out the step-by-step reasoning and logical deductions as they appear in the source. Your goal is to make the internal logic of the lecture clear and traceable for revision. Do not include any information, examples, or intuition that is not explicitly found in the provided slides.

Below is a **faithful, exam-ready lecture scribe** for **CSE400 – Lecture 10: Randomized Min-Cut Algorithm**, prepared **strictly from the provided slides**.

No additional intuition, examples, or external material has been introduced.

All definitions, statements, algorithms, and explanations follow the **exact logical flow and wording of the source**, with **step-by-step reasoning explicitly written out** to make the internal logic traceable during revision.

CSE400 – Fundamentals of Probability in Computing

Lecture 10: Randomized Min-Cut Algorithm

1. Min-Cut Problem

1.1 Why Use Min-Cut?

The **min-cut algorithm** is used in various applications to solve problems related to:

1. **Network connectivity**
2. **Network reliability**
3. **Optimization**

The motivation is explained through specific applications:

- **Network Design**

Min-cut helps in improving the efficiency of communication and optimizing network flow.

The algorithm is used in network design to find the **minimum capacity cut**.

- **Communication Networks**

Min-cut is useful for understanding the **vulnerability of networks to failures**.

It helps in building **robust and fault-tolerant communication networks**.

- **VLSI Design**

In Very Large Scale Integration (VLSI) design, min-cut is useful for **partitioning circuits into smaller components**, which leads to **reduced interconnectivity complexity**.

2. What is Min-Cut?

2.1 Cut-Set

Definition (Cut-Set):

A *cut-set* in a graph is a **set of edges whose removal breaks the graph into two or more connected components**.

2.2 Minimum Cut (Min-Cut)

Given:

- A graph $G = (V, E)$
- $n = |V|$ vertices

Definition (Min-Cut Problem):

The **minimum cut**, or **min-cut**, problem is to find a **minimum cardinality cut-set** in the graph G .

That is:

- Among all possible cut-sets,
 - Choose the one with the **smallest number of edges**.
-

2.3 Sensitivity of Min-Cut Algorithms

Min-cut algorithms such as **Karger's algorithm** are:

- **Randomized**
- **Sensitive to the initial choice of edges**

Logical implication:

- If the algorithm **contracts critical edges early**,
 - Then the resulting cut may be **smaller or incorrect**, depending on randomness.
-

3. Edge Contraction

3.1 Main Operation

The **main operation** in the min-cut algorithm is **edge contraction**.

Definition (Edge Contraction):

Edge contraction is an operation that:

1. Removes an edge from the graph, and
 2. **Simultaneously merges the two vertices** connected by that edge.
-

3.2 Contracting an Edge (u, v)

When contracting an edge (u, v) :

Step-by-step:

1. Vertices u and v are **merged into a single vertex**.
2. All edges **connecting u and v** are eliminated.
3. All **other edges** in the graph are retained.

Properties of the resulting graph:

- Parallel edges **may exist**
 - **Self-loops do not exist**
-

4. Successful and Unsuccessful Min-Cut Runs

4.1 Successful Min-Cut Run

Definition:

A **successful min-cut run** refers to the success in the **outcome of an algorithm designed to find the minimum cut** in a graph.

Logical meaning:

- The algorithm correctly identifies the **true minimum cut**.

(Referenced visually by the “Successful Run” figure in the slides.)

4.2 Unsuccessful Min-Cut Run

Definition:

An **unsuccessful min-cut run** refers to an iteration of a min-cut algorithm where the algorithm **fails to correctly identify the minimum cut** of a given graph.

Logical meaning:

- Due to poor random choices (e.g., contracting critical edges),
- The algorithm outputs a cut that is **not minimum**.

(Referenced visually by the “Unsuccessful Run” figure in the slides.)

5. Max-Flow Min-Cut Theorem

5.1 Statement of the Theorem

Max-Flow Min-Cut Theorem:

In a flow network, the maximum amount of flow passing from the source to the sink is equal to the total weight of the edges in a minimum cut.

5.2 Related Definitions

Let a cut divide the vertex set into two parts X and Y :

- **Capacity of a cut**
The sum of the capacities of edges oriented from a vertex in X to a vertex in Y .
- **Minimum cut**
The cut in the network that has the **smallest possible capacity**.

- **Minimum cut capacity**
The capacity value of the minimum cut.
- **Maximum flow**
The largest possible flow from the **source** S to the **sink** T .

Logical implication of the theorem:

- Computing maximum flow also determines the minimum cut capacity.
 - Both quantities are **equal**.
-

6. Deterministic Min-Cut Algorithm

Stoer–Wagner Min-Cut Algorithm

6.1 Theorem Used

Let:

- s and t be two vertices of a graph G
- $G/\{s, t\}$ be the graph obtained by **merging vertices s and t**

Statement:

A minimum cut of G can be obtained by taking the **smaller** of:

1. A minimum s - t cut of G , and
 2. A minimum cut of $G/\{s, t\}$
-

6.2 Reasoning Behind the Theorem

The theorem holds because **exactly one of the following cases must occur**:

1. There exists a minimum cut of G that **separates s and t**
→ Then the minimum s - t cut of G is a minimum cut of G .
 2. There is **no minimum cut** that separates s and t
→ Then the minimum cut of the merged graph $G/\{s, t\}$ is a minimum cut of G .
-

6.3 Pseudocode

Algorithm 1: MinimumCutPhase(G , a)

1. Initialize $A \leftarrow \{a\}$
2. While $A \not\equiv V$:

- Add to A the **most tightly connected vertex**
3. Return the cut weight as the **cut of the phase**
-

Algorithm 2: `MinimumCut(G)`

1. While $|V| \geq 1$:
 - Choose any $a \in V$
 - Run `MinimumCutPhase(G, a)`
 2. If the **cut of the phase** is lighter than the current minimum cut:
 - Store it as the current minimum cut
 3. Shrink G by **merging the last two vertices added**
 4. Return the **minimum cut**
-

7. Randomized Min-Cut Algorithm

7.1 Why Randomized Algorithms?

Randomized algorithms:

1. Provide a **probabilistic guarantee of success**
2. Provide a **more accurate estimate** of the minimum cut with **fewer iterations**

Additional reasons listed:

- Efficiency
 - Parallelization
 - Approximation guarantees
 - Avoidance of worst-case instances
 - Heuristic nature
 - Robustness
-

8. Karger's Randomized Algorithm

Karger's algorithm repeatedly:

1. Randomly selects an edge
2. Contracts the selected edge
3. Continues until only **two vertices remain**
4. The remaining edges define a cut

The output may or may not be the true minimum cut, depending on randomness.

8.1 Recursive Randomized Min-Cut – Pseudocode

Algorithm 3: `RECURSIVE-RANDOMIZED-MIN-CUT( $G, \alpha$ )`

Input:

- Undirected multigraph G with n vertices
- Integer constant $\alpha > 0$

Output:

- A cut C of G

Steps:

1. If $n \leq 3$:
 - Compute min-cut using **brute-force (exhaustive) search**
 - Set $C \leftarrow$ min-cut found
 2. Else:
 - For $i = 1$ to α :
 1. Construct G' by applying $n - \lceil n/\sqrt{\alpha} \rceil$ random contraction steps on G
 2. Compute $C' \leftarrow$ `RECURSIVE-RANDOMIZED-MIN-CUT( $G', \alpha$ )`
 3. If $i = 1$ or $|C'| < |C|$:
 - Set $C \leftarrow C'$
 3. Return C
-

9. Deterministic vs Randomized Min-Cut

9.1 Deterministic Min-Cut

- Always guarantees an **exact minimum cut**
- May have **higher time complexity** for large graphs
- Stoer–Wagner algorithm complexity:

$$O(V \cdot E + V^2 \log V)$$

9.2 Randomized Min-Cut

- Provides an **approximate minimum cut with high probability**
- Karger's algorithm complexity:

$$O(V^2)$$

10. Theorem for Min-Cut Set

Theorem:

The algorithm outputs a **minimum cut set** with probability at least:

$$\frac{2}{n(n-1)}$$

11. Python Simulation

Students are instructed to:

1. Open the **Campuswire post** for Lecture 10
 2. Download the `.ipynb` **file** shared in the comments
-

End of Lecture 10

Thank You

If you want, I can next:

- Convert this **exact scribe into LaTeX**, or
- Split it into **exam-wise short notes + derivation blocks**, or
- Prepare a **probability analysis of Karger's theorem strictly from slides**.